Getting Started with the Graph OLAP SDK

This guide helps you install and configure the Graph OLAP Python SDK for Jupyter notebook-based graph analytics.

Introduction

What is Graph OLAP Platform?

Graph OLAP Platform is an enterprise analytics solution that enables you to:

- **Transform relational data into graphs**: Define mappings from your Starburst data warehouse tables to graph structures with nodes and relationships.
- **Create point-in-time snapshots**: Export data from Starburst into optimized graph format for analysis.
- **Run graph instances**: Spin up in-memory graph databases loaded with your snapshot data.
- **Query and analyze**: Execute Cypher queries and run graph algorithms (PageRank, community detection, shortest paths, and more).

What is the Jupyter SDK?

The Graph OLAP SDK (`graph-olap-sdk`) is the **sole user interface** for the Graph OLAP Platform. All platform operations - from creating mappings to running graph algorithms - are performed through this Python SDK in Jupyter notebooks. There is no separate web console or GUI; the SDK is the complete interface for analysts.

The SDK provides:

Capability	Description
Mapping Management	Full CRUD operations: create, read, update, delete, copy, and list mappings

Capability	Description
Instance Lifecycle	Create instances from mappings, terminate, update CPU, monitor status
Graph Queries	Execute Cypher queries with DataFrame results (Polars/Pandas)
Graph Algorithms	Run native algorithms (PageRank, Louvain) and 500+ NetworkX algorithms
Schema Discovery	Browse Starburst catalogs, schemas, tables, and columns to design mappings
Visualization	Interactive graph visualization with PyVis and Plotly
Admin Operations	Bulk delete, cluster health monitoring, configuration (role-based)
Zero-Config Setup	Auto-discovery from environment variables in JupyterHub

> **Note:** The SDK is notebook-first by design. This ensures all operations are reproducible, version-controllable, and seamlessly integrated with data science workflows.

Key Concepts

Before diving into the SDK, understand these core concepts:

```
┌──────────────┐      ┌──────────────┐      ┌──────────────┐
│   Mapping    │──────►│  Instance    │──────►│   Query      │
│  (Schema)    │      │  (Runtime)   │      │  (Analysis)  │
└──────────────┘      └──────────────┘      └──────────────┘
```

- **Mapping**: Defines which tables become nodes and which become relationships.
- **Instance**: A running graph database created from a mapping (snapshot managed internally).
- **Connection**: A handle to an instance for executing Cypher queries and algorithms.

> **Note:** Snapshots are managed internally when creating instances. Use `client.instances.create_from_mapping()` to create instances directly from mappings.

Installation Options

The SDK offers tiered installation options to match your needs:

Option	Dependencies	Command	Use Case
Minimal	httpx, pydantic, tenacity	`pip install graph-olap-sdk`	API operations only
Analyst	+ polars, pandas, pyvis, plotly, networkx, itables	`pip install graph-olap-sdk[all]`	Full notebook analytics
Interactive	+ ipywidgets	`pip install graph-olap-sdk[all,interactive]`	Widget-based UI

Minimal Installation

For environments where you only need API operations without DataFrame or visualization support:

```
pip install graph-olap-sdk
```

Core dependencies:

- `httpx>=0.28.1` - HTTP client with connection pooling
- `pydantic>=2.12.5` - Data validation and serialization
- `tenacity>=9.1.2` - Retry logic for transient failures
- `graph-olap-schemas>=1.0.0` - Shared type definitions

Analyst Installation (Recommended)

For full notebook analytics with DataFrames and visualization:

```
pip install graph-olap-sdk[all]
```

Additional dependencies:

- `polars>=1.36.1` - Fast DataFrame library (default)
- `pandas>=2.3.3` - Traditional DataFrame support
- `networkx>=3.6.1` - Graph analysis library
- `pyvis>=0.3.2` - Interactive graph visualization
- `plotly>=6.5.0` - Interactive charts and graphs
- `itables>=2.6.2` - Interactive DataFrames in notebooks

Verifying Installation

After installation, verify the SDK is working:

```
import graph_olap
print(f"Graph OLAP SDK version: {graph_olap.__version__}")
```

Quick Start Example

This section demonstrates the most common workflow: connecting to the platform, creating resources, and running queries.

Simplest Approach: Zero-Config Notebook

If you're in a JupyterHub environment with pre-configured environment variables:

```
from graph_olap import notebook

# Auto-discovers configuration from environment
```

```
client = notebook.connect()

# List available mappings
mappings = client.mappings.list()
for mapping in mappings.items:
    print(f"- {mapping.name} (ID: {mapping.id})")
```

Explicit Configuration

For explicit control over connection settings:

```
from graph_olap import GraphOLAPClient

# Create client with explicit configuration
client = GraphOLAPClient(
    api_url="https://graph-olap.example.com",
    api_key="your-api-key-here",
)

# Or load from environment with overrides
client = GraphOLAPClient.from_env(
    timeout=60.0,      # Override default 30s timeout
    max_retries=5,     # Override default 3 retries
)
```

Complete Workflow Example

Here's a complete workflow from mapping to analysis:

```
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType

# 1. Connect to the platform
client = GraphOLAPClient.from_env()

# 2. Find an existing mapping
mappings = client.mappings.list(search="customer")
mapping = mappings.items[0]
print(f"Using mapping: {mapping.name}")

# 3. Create an instance directly from mapping (snapshot managed internally)
instance = client.instances.create_from_mapping_and_wait(
    mapping_id=mapping.id,
    name="Customer Analysis Instance",
    wrapper_type=WrapperType.FALKORDB,  # or WrapperType.RYUGRAPH
```

```
    ttl=24,  # Auto-terminate after 24 hours
    timeout=600,  # Wait up to 10 minutes for export + startup
)
print(f"Instance running: {instance.id}")

# 4. Connect and query
conn = client.instances.connect(instance.id)

# Simple query
result = conn.query("MATCH (n:Customer) RETURN n.name, n.id LIMIT 10")
print(result.to_polars())

# 5. Run algorithms
conn.algo.pagerank(
    node_label="Customer",
    property_name="influence_score",
    damping=0.85,
)

# Query the results
top_customers = conn.query_df(
    "MATCH (c:Customer) "
    "RETURN c.name, c.influence_score "
    "ORDER BY c.influence_score DESC "
    "LIMIT 10"
)
print(top_customers)

# 6. Clean up when done
client.instances.terminate(instance.id)
client.close()
```

Using Quick Start Helper

For rapid prototyping, use the `quick_start` method:

```
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType

client = GraphOLAPClient.from_env()

# Creates instance from mapping in one call (snapshot managed internally)
conn = client.quick_start(
    mapping_id=1,
    wrapper_type=WrapperType.FALKORDB,
    instance_name="Quick Instance",
)

# Ready to query immediately
count = conn.query_scalar("MATCH (n) RETURN count(n)")
```

```
print(f"Total nodes: {count}")

# Remember to terminate when done!
client.instances.terminate(conn.instance_id)
```

Authentication Setup

The SDK supports multiple authentication modes for different environments.

Environment Variables

The recommended approach is to use environment variables:

Variable	Description	Required
`GRAPH_OLAP_API_URL`	Base URL for the control plane API	Yes
`GRAPH_OLAP_API_KEY`	API key for Bearer token authentication	Production
`GRAPH_OLAP_INTERNAL_API_KEY`	Internal API key (X-Internal-Api-Key header)	Internal services
`GRAPH_OLAP_USERNAME`	Username for development/testing	Development

Example `.env` file:

```
GRAPH_OLAP_API_URL=https://graph-olap.example.com
GRAPH_OLAP_API_KEY=sk-your-api-key-here
```

Authentication Modes

The SDK supports three authentication modes with clear priority rules:

1. API Key (Production)

Standard Bearer token authentication for production environments:

```
client = GraphOLAPClient(
    api_url="https://graph-olap.example.com",
    api_key="sk-your-api-key",
)
# Sends: Authorization: Bearer sk-your-api-key
```

2. Internal API Key (Service-to-Service)

For internal services communicating within the platform:

```
client = GraphOLAPClient(
    api_url="https://graph-olap.internal",
    internal_api_key="internal-service-key",
)
# Sends: X-Internal-Api-Key: internal-service-key
```

3. Username Header (Development)

For local development and testing without authentication gateway:

```
client = GraphOLAPClient(
    api_url="http://localhost:8000",
    username="test-user",
    role="analyst",  # Optional: analyst, admin, or ops
)
# Sends: X-Username: test-user
# Sends: X-User-Role: analyst
```

Authentication Priority:

When multiple credentials are provided:

1. `internal_api_key` takes precedence over `api_key`
2. `username` is always sent if provided (in addition to API key)

JupyterHub Zero-Config

In JupyterHub environments, the SDK auto-discovers configuration:

```python
from graph_olap import notebook

# Environment variables are pre-configured by JupyterHub
client = notebook.connect()
```

The JupyterHub deployment automatically injects:

- `GRAPH_OLAP_API_URL` pointing to the control plane
- `GRAPH_OLAP_API_KEY` with user-specific credentials

Production Environment Notes

> **Important**: In production environments with authentication gateways (IAP/OIDC), the `username` and `role` parameters are **ignored**. The gateway strips user-supplied headers and injects validated identity from the authenticated session. These parameters are only effective in local development and E2E testing.

Query Methods Reference

Once connected to an instance, you have several query methods available:

Method	Returns	Use Case
`query()`	`QueryResult`	Full result with multiple format options
`query_df()`	DataFrame	Direct DataFrame (Polars or Pandas)
`query_scalar()`	Single value	COUNT, SUM, or single-value queries
`query_one()`	Dict or None	Single row as dictionary

Query Examples

```python
conn = client.instances.connect(instance_id)

# Full QueryResult with format options
result = conn.query("MATCH (n:Customer) RETURN n.name, n.age LIMIT 100")
```

```
polars_df = result.to_polars()
pandas_df = result.to_pandas()

# Direct DataFrame
df = conn.query_df(
    "MATCH (n)-[r]->(m) RETURN n.name, type(r), m.name",
    backend="polars",  # or "pandas"
)

# Single scalar value
count = conn.query_scalar("MATCH (n:Customer) RETURN count(n)")

# Single row as dict
customer = conn.query_one(
    "MATCH (c:Customer {id: $id}) RETURN c.name, c.email",
    parameters={"id": "C001"}
)
```

Using Parameters

Always use parameters for dynamic values to prevent injection:

```
# Good: Using parameters
result = conn.query(
    "MATCH (c:Customer) WHERE c.region = $region RETURN c",
    parameters={"region": "APAC"}
)

# Bad: String interpolation (vulnerable to injection)
region = "APAC"
result = conn.query(f"MATCH (c:Customer) WHERE c.region = '{region}' RETURN c")
```

Error Handling

The SDK raises specific exceptions for different error conditions:

```
from graph_olap.exceptions import (
    NotFoundError,
    ValidationError,
    InvalidStateError,
    TimeoutError,
    AuthenticationError,
```

```
)

try:
    instance = client.instances.get(999)
except NotFoundError:
    print("Instance not found")
except AuthenticationError:
    print("Invalid API key")
except ValidationError as e:
    print(f"Invalid request: {e}")
```

Common Exceptions

Exception	HTTP Status	Description
`NotFoundError`	404	Resource doesn't exist
`ValidationError`	400/422	Invalid request parameters
`AuthenticationError`	401	Invalid or missing credentials
`PermissionDeniedError`	403	Insufficient permissions
`InvalidStateError`	409	Resource in wrong state for operation
`TimeoutError`	-	Operation exceeded timeout
`InstanceFailedError`	-	Instance startup failed
`SnapshotFailedError`	-	Snapshot creation failed

Next Steps

Now that you have the SDK installed and configured, explore these topics:

Topic	Document	Description
Core Concepts	02-core-concepts.manual.md	Resource hierarchy and workflows
API Reference	03-api-reference.manual.md	Full SDK API documentation
Advanced Topics	05-advanced-topics.manual.md	Connection management, error handling
Examples	06-examples.manual.md	Complete workflow examples

Getting Help

If you encounter issues:

1. Check the error message for specific guidance
2. Review environment variable configuration
3. Verify network connectivity to the API URL
4. Check instance status before querying

Appendix: Environment Variable Reference

Complete list of environment variables recognized by the SDK:

Variable	Default	Description
`GRAPH_OLAP_API_URL`	*required*	Control plane API base URL
`GRAPH_OLAP_API_KEY`	None	API key for Bearer authentication
`GRAPH_OLAP_INTERNAL_API_KEY`	None	Internal service API key
`GRAPH_OLAP_USERNAME`	None	Username header for development

Variable	Default	Description
`GRAPH_OLAP_IN_CLUSTER_MODE`	false	Use in-cluster DNS for wrapper connections
`GRAPH_OLAP_NAMESPACE`	e2e-test	Kubernetes namespace for in-cluster mode
`GRAPH_OLAP_SKIP_HEALTH_CHECK`	false	Skip wrapper health checks (port-forward mode)

Document version: 1.0.0 | Last updated: 2025-01